

Transformers and Self-Attention

The Core Mechanism

Transformers rely on the Self-Attention mechanism to weigh the importance of different words in a sequence regardless of their distance. This allows for massive parallelization compared to RNNs.

PyTorch Multi-Head Attention

```
import torch
import torch.nn as nn

# Parameters: embed_dim=512, num_heads=8
multihead_attn = nn.MultiheadAttention(embed_dim=512, num_heads=8)

query = torch.randn(10, 32, 512) # (L, N, E)
key = torch.randn(10, 32, 512)
value = torch.randn(10, 32, 512)

attn_output, attn_weights = multihead_attn(query, key, value)
print(attn_output.shape) # torch.Size([10, 32, 512])
```

Syntax & Debugging

Syntax: `embed_dim` must be divisible by `num_heads`. Debugging Tip: Check for 'NaN' losses often caused by exploding gradients in deep attention layers; use Gradient Clipping.

Summary: Transformers use self-attention to process sequences in parallel, forming the backbone of modern LLMs.

Generative Adversarial Networks (GANs)

The Minimax Game

GANs consist of two networks: a Generator that creates fake data and a Discriminator that attempts to distinguish between real and fake data. They are trained simultaneously in a competitive process.

GAN Training Loop Snippet

```
# discriminator loss
d_loss_real = criterion(discriminator(real_images), real_labels)
d_loss_fake = criterion(discriminator(generator(z)), fake_labels)
d_loss = d_loss_real + d_loss_fake

# generator loss
g_loss = criterion(discriminator(generator(z)), real_labels)
```

Best Practices

Use 'LeakyReLU' instead of 'ReLU' to prevent dying neurons. Use 'Batch Normalization' in both networks to stabilize training. Avoid 'Mode Collapse' by using Wasserstein loss (WGAN).

Summary: GANs use a zero-sum game between a generator and discriminator to produce highly realistic synthetic data.

Text Generation and Decoding

Autoregressive Sampling

Models like GPT-4 generate text token-by-token. The strategy used to pick the next token (Decoding) significantly impacts the quality and creativity of the output.

Hugging Face Decoding

```
from transformers import pipeline

gen = pipeline("text-generation", model="gpt2")
output = gen("The future of AI is",
max_length=30,
do_sample=True,
top_k=50,
temperature=0.7)
print(output[0]['generated_text'])
```

Syntax Explanations

Temperature: Scales logits before softmax (lower is more deterministic). Top-k: Limits sampling to the top k most likely tokens. Top-p (Nucleus): Limits sampling to a cumulative probability threshold.

Summary: Text generation relies on autoregressive decoding where temperature and sampling methods control the balance between logic and creativity.

Visual Synthesis: Diffusion Models

Reverse Diffusion Process

Diffusion models work by gradually adding Gaussian noise to an image until it becomes pure noise, then learning to reverse this process to recover the original image from noise.

Stable Diffusion with Diffusers

```
from diffusers import StableDiffusionPipeline
import torch

model_id = "runwayml/stable-diffusion-v1-5"
pipe = StableDiffusionPipeline.from_pretrained(model_id,
torch_dtype=torch.float16)
pipe = pipe.to("cuda")

prompt = "A futuristic city in the style of Van Gogh"
image = pipe(prompt).images[0]
image.save("output.png")
```

Debugging Tips

If images are black or blurry: 1. Check if the safety checker is triggering. 2. Ensure your prompt isn't too short. 3. Verify that you are using the correct scheduler (e.g., Euler, PNDM).

Summary: Diffusion models have surpassed GANs in image quality by iteratively refining noise into structured visuals.

Audio and Music Generation

Waveform vs. Spectrogram

Audio can be generated directly as raw waveforms (WaveNet) or as 2D spectrograms that are later converted to sound via a Vocoder.

Simple Audio Wave Synthesis

```
import numpy as np
import scipy.io.wavfile as wav

sample_rate = 44100
duration = 2.0 # seconds
frequency = 440.0 # A4 note

t = np.linspace(0, duration, int(sample_rate * duration))
audio_data = 0.5 * np.sin(2 * np.pi * frequency * t)

# Normalize to 16-bit PCM
audio_data = (audio_data * 32767).astype(np.int16)
wav.write("synth_tone.wav", sample_rate, audio_data)
```

Best Practices

Always use a consistent sample rate (e.g., 22050Hz or 44100Hz) across your dataset. Use 'Phase-Vocoders' or 'HiFi-GAN' for high-quality audio reconstruction from mel-spectrograms.

Summary: Generative audio involves predicting sequential samples or spectral features to synthesize speech and music.

Hallucinations and Factuality

The Hallucination Problem

Hallucination occurs when a model generates confident but false information. This is often due to the model's objective (predicting the next token) rather than 'knowing' facts.

RAG Implementation (Simplified)

```
def generate_with_rag(query, context_docs):  
    # Combine context with query to ground the model  
    prompt = f"Context: {context_docs}\n\nQuestion: {query}\n\nAnswer:"  
    response = llm.generate(prompt)  
    return response  
  
# Debugging: If model ignores context, use 'System Prompts' to enforce  
# strictness.
```

Mitigation Best Practices

1. Retrieval-Augmented Generation (RAG). 2. Chain-of-Thought Prompting. 3. Fact-checking benchmarks (e.g., TruthfulQA). 4. Human-in-the-loop verification.

Summary: Factuality is a major challenge; grounding models with external data via RAG is the most effective current solution.